



Good
Evening

Everyone ☺

TRANSPOSE OF A SQUARE MATRIX

The transpose of a matrix is new matrix obtained by interchanging the rows & columns of the original matrix

$$\begin{array}{ccc} \text{mat}[3][3] \Rightarrow \{ & & \{ \\ & \begin{array}{c} \text{0} \{ 1, 2, 3 \} \\ \text{(1,0)} \rightarrow 1 \{ 5, 6, 7 \} \\ \text{(2,0)} \leftarrow 2 \{ 9, 10, 11 \} \end{array} & \Rightarrow \{ \\ & & \{ 1, 5, 9 \} \\ & & \{ 2, 6, 10 \} \\ & & \{ 3, 7, 11 \} \end{array}$$

Diagram illustrating the transpose of a 3x3 matrix. The original matrix is shown on the left, with rows and columns labeled. The first row is {1, 2, 3}, the second row is {5, 6, 7}, and the third row is {9, 10, 11}. The first column is {1, 5, 9}, the second column is {2, 6, 10}, and the third column is {3, 7, 11}. The transpose operation is indicated by an arrow pointing to the right, where the resulting transposed matrix is shown. The first row of the transposed matrix is {1, 5, 9}, the second row is {2, 6, 10}, and the third row is {3, 7, 11}.

Approach

(0,1)

	0	1	2	3	4
0	1	6	11	16	21
1	2	7	12	17	22
2	3	8	13	14	15
3	4	9	18	19	20
4	5	10	23	24	25

0th row $\rightarrow$ 0th col
1th row $\rightarrow$ 1st col
2nd row $\rightarrow$ 2nd col

1	6	11	16	21
2	7	12	17	22
3	8	13	18	23
4	9	14	19	24
5	10	15	20	25

✓
TC: $O(N^2)$
SC: $O(1)$

function findTranspose (matrix[N][N]) {

for (i $\rightarrow$ 0 to N-1) {

for (j $\rightarrow$ i+1 to N-1) {

swap (matrix[i][j], matrix[j][i])

}

}

}

ROTATE A MATRIX TO 90 DEGREE CLOCKWISE

1	2	3
4	5	6
7	8	9

7	4	1
8	5	2
9	6	3

1	2	3	4	5
6	7	8	9	10
11	12	13	14	15
16	17	18	19	20
21	22	23	24	25

1	6	11	16	21
2	7	12	17	22
3	8	13	18	23
4	9	14	19	24
5	10	15	20	25

→ reverse

→ reverse

→ reverse

→ reverse

→ reverse

21	16	11	6	1
22	17	12	7	2
23	18	13	8	3
24	19	14	9	4
25	20	15	10	5

Calculate transpose & Reverse Every Row = Rotate by 90°

Approach:

TC: $O(N^2)$
SC: $O(1)$

function rotate (mat[N][N]) {

mat = transpose (mat)

for (i → 0 to N-1) {

reverse (mat[i])

}

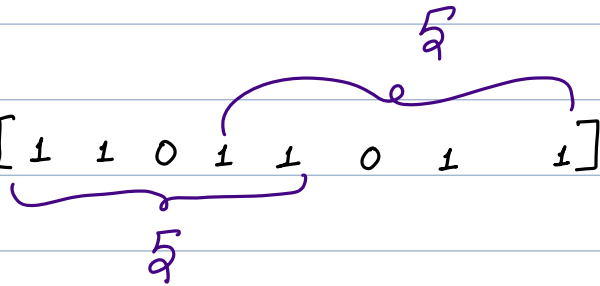
return mat

}

FIND THE MAXIMUM NUMBER OF CONSECUTIVE 1's AFTER REPLACEMENT

Given an array of 1s and 0s, you are allowed to replace only one 0 with 1. Find the maximum number of consecutive 1s that can be obtained after making replacement.

Input :- $[1 \ 1 \ 0 \ 1 \ 1 \ 0 \ 1 \ 1]$



Ans = 5

Quiz 2.

Arr = $[1 \ 1 \ 0 \ 1 \ 1 \ 0 \ 1 \ 1 \ 1]$

$\Rightarrow$ 6

$[0 \ 1 \ 1 \ 1 \ 1 \ 1 \ 1]$

total Ones
 $\Rightarrow 6$

2N

$\overbrace{1 \ 1 \ 1 \ 1 \ 1}^0 \textcircled{0}$

4

3N

$\Rightarrow \underline{O(N)}$

Quiz 2.

Ans = $[0 \quad 1 \quad 1 \quad 1 \quad 0 \quad 1 \quad 1 \quad 0 \quad 1 \quad 1 \quad 0]$

$[0 \quad 0 \quad 1 \quad 1 \quad 0]$

Code:-

function findMaxConsecutiveOnes (nums []) {

n = nums.length;

maxcount = 0

totalOnes = 0

for (i = 0 to n-1)
 if (nums[i] == 1)
 totalOnes++
 }

if (totalOnes == n)
 return n
 }

```
for (i = 0 to n-1) {  
    if (nums[i] == 0) {
```

```
        int l = 0, r = 0, j = i+1
```

```
        // find consecutive 1's at right
```

```
        while (j < n and nums[j] == 1)  
            r++  
            j++  
        }
```

```
        // find consecutive 1's at left
```

```
        j = i-1
```

```
        while (j >= 0 and nums[j] == 1)  
            l++  
            j--  
        }
```

```
        count = l + r + 1
```

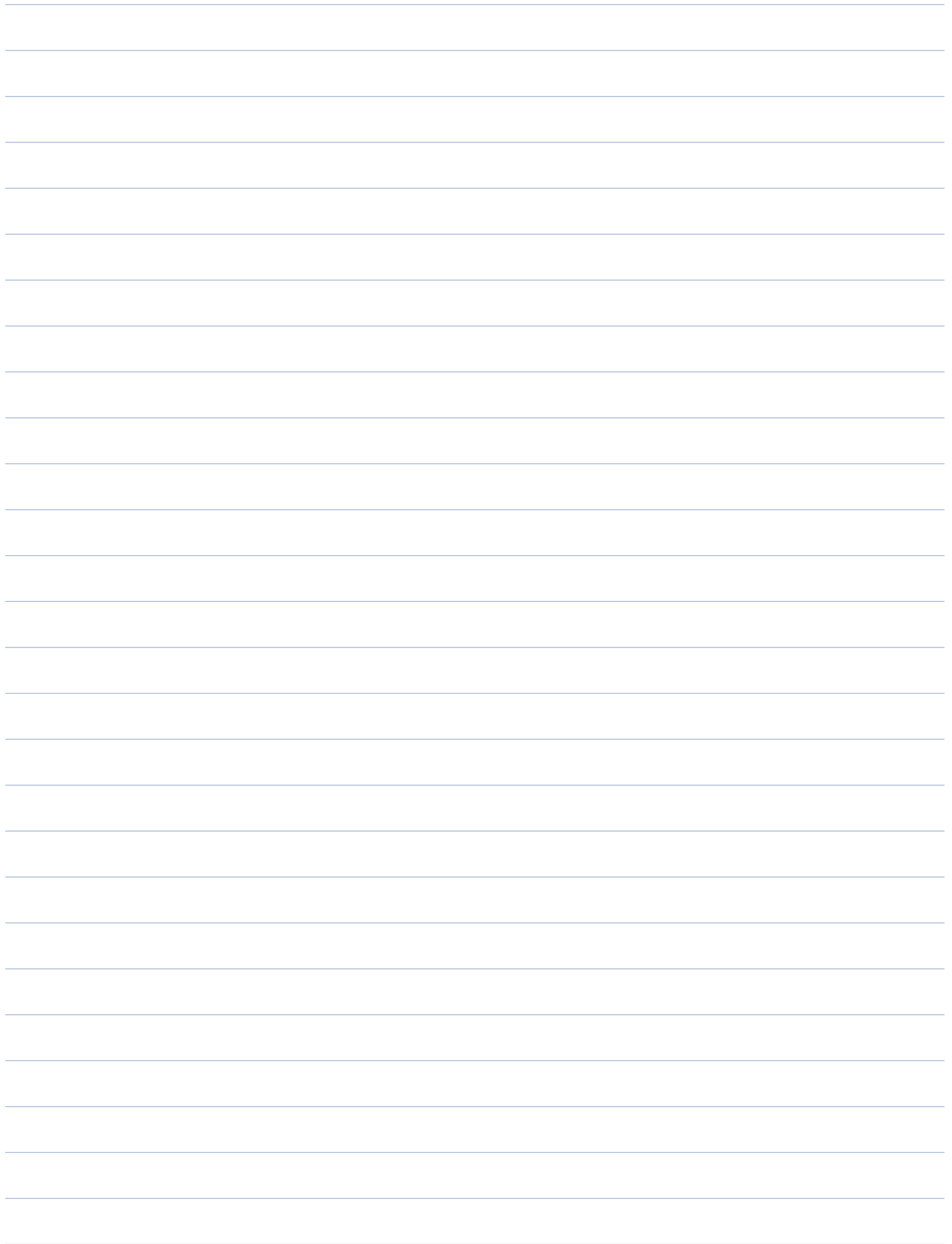
```
        if (count > maxCount)
```

```
            maxCount = count
```

```
    }
```

```
    print (maxCount)
```

```
}
```

CONSECUTIVE ONES VARIATION

Given an array of 1s and 0s, you are allowed to replace only one 0 with 1. Find the maximum number of consecutive 1s that can be obtained by SWAPPING at most one 0 with 1 (already present in string)

$\Rightarrow 4$

Input $\rightarrow$ $[1 \ 0 \ 1 \ 1 \ 0 \ 1]$ $\Rightarrow \underline{4}$

4

ans = ~~3~~ 4

Total $\Rightarrow 6$

$[1 \ 1 \ 0 \ 0 \ 1 \ 1 \ 0 \ 1 \ 1 \ 0]$

$l = 2$
 $r = 2$

$l + r + 1$
 $2 + 2 + 1$

Ques 2

$[1 \ 1 \ 0 \ 1 \ 1 \ 1]$

$\Rightarrow \underline{5}$

Code :

TC : $O(N)$

SC : $O(1)$

function findMaxConsecutive (nums[]) {

n = nums.size

maxcount = 0

totalOnes = 0

```
for (i = 0 to n-1)
    if (nums[i] == 1)
        totalOnes ++
    }
```

```
if (totalOnes == n)
    return n
```

```
for (i = 0 to n-1) {
```

```
    if (nums[i] == 0) {
```

```
        l = 0, r = 0, j = i + 1
```

```
        while (j < n && nums[j] == 1) {
```

```
            r++
```

```
            j++
```

```
        }
```

j = i-1

```
while (j >= 0 && nums[j] == 1)
    j++
    j--
```

}

```
if (l+r == totalOnes)
```

curmax = l+r

}

```
else {
```

curmax = l+r+1

}

maxcount = max(curmax, maxcount)

}

}

REVERSE STRING WORD BY WORD

Given an input string s , reverse the order of words. A word is defined as sequence of non-space characters. The words in " s " are separated by atleast one space.

Return a string of words in reverse order, concatenated by single space. The returned string should have single space separating the words.

String s = "My Name is Twinkle"

"elkniwT si eman yM"

"Twinkle is Name My"

s = "Hello World"

World Hello

"Hello World"

// Trim the Spaces

- ↳ Spaces at left
- ↳ Spaces from right
- ↳ Ignore inbetween spaces .

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22
" Scaler is best "

// Reverse the entire string

// Reverse each word in the string.